

REPORT

정규 표현식 정리 보고서



과 목 명: AI응용과 이해

지도교수: 이 혜 정 교수님

학 과: 인공지능 소프트웨어

이 름: 김 근 하

제 출 일: 2026.04.28



목 차

1. 개 요

2. 구성 요소 및 사용 방식

1. 개요 (Introduction)

■ 정규 표현식이란?

- 문자열의 패턴을 표현하는 특별한 문법이다. - 점프 투 파이썬 -
- 프로그래밍에서 문자열을 다룰 때, 문자열의 일정한 패턴을 표현하는 일종의 형식 언어를 말함. 다른 말로 정규식 또는 Regex 라고도 불린다.
- 정규표현식은 많은 텍스트 편집기와 프로그래밍 언어에서 문자열과 검색, 치환을 위해 지원하고 있으며, 특히 펄 과 TCL은 언어 자체에 정규 표현식을 구현하고 있다.

2. 구성 요소 및 사용 방식

■ 메타 문자

- 정규 표현식에서 사용 될 때, 특별한 의미를 가지는 문자를 뜻함.

- ① 문자 클래스 [] : 이 중에서 아무거나 하나 (여러 개의 선택지 중에 하나를 고르는 것)

예시) [ABC] : A,B,C 중에 하나의 문자만 찾는 것 (이중에 하나의 문자만 찾음)

- ② 하이픈(-) 사용 : [] 안에 두 문자 사이에 '-' 를 사용하면 두 문자 사이의 범위를 간편하게 표현할 수 있다. 연속된 문자들을 일일이 나열하지 않아도 된다.

예시) [a-c] → a 부터 c 까지 a, b, c

[a-zA-Z] → 모든 알파벳 (대문자와 소문자 모두)

- ③ ^ 사용법 : 문자 클래스 앞에 ^를 사용하면 "~가 아닌" 이라는 의미가 된다. 해당 문자 제외 모든 문자와 매치 됨.

예시) [^abc] → a, b, c 가 아닌 모든 문자

- ④ . (점) 문자 : 줄바꿈 문자인 $\backslash n$ 을 제외한 모든 문자와 매치된다는 것을 의미한다.

예시 1) a.b → "a.b" 문자열과 "a0b" 문자열매치

예시 2) a.[b] → a.b" 문자열과 매치되고 "a0b" 문자열과는 매치되지 않는다

REPORT

- ⑤ * 문자 : * 바로 앞에 있는 문자 a가 0부터 무한대까지 반복될 수 있다는 의미 (실제로는 시스템 메모리, 입력 데이터의 크기 등에 의해 제한 됨)

예시) $ct \leftrightarrow ca^*t \rightarrow a$ 가 0번 반복되어 매치, $caa^*t \leftrightarrow ca^*t \rightarrow a$ 가 3번 반복되어 매치

- ⑥ + 문자: +앞에 있는 문자가 1번부터 반복 됨

예시) $ct \leftrightarrow ca+t \rightarrow a$ 가 0번 반복되어 매칭되지 않음.

$caa^+t \leftrightarrow ca+t \rightarrow a$ 가 3번 반복되어 매치 (3번 반복)

- ⑦ {} 문자 : 앞에 있는 문자의 반복횟수를 n 부터 m 까지 만 제한 하고 싶을 때 사용.

예시) $ca\{2,5\}t \rightarrow a$ 를 2~5 회 반복

$ca\{2\}t \rightarrow a$ 를 반드시 2번만 반복

$\{1,\}$ $\rightarrow +$ 와 동일

$\{0,\}$ $\rightarrow *$ 와 동일

- ⑧ ? 문자 : 앞에 문자에 대한 조건 식으로 "있어도 되고 없어도 상관 없음."

예시) $ct \leftrightarrow ca?t \rightarrow a$ 가 0번 사용되어 매치

$caa^?t \leftrightarrow ca?t \rightarrow a$ 가 3번 사용되어 매치

=====※ 자주 사용하는 문자 클래스 ※=====

$\backslash d$ - 숫자와 매치된다. $[0-9]$ 와 동일한 표현식이다.

$\backslash D$ - 숫자가 아닌 것과 매치된다. $[\wedge 0-9]$ 와 동일한 표현식이다.

$\backslash s$ - 화이트스페이스(whitespace) 문자와 매치된다. $[\wedge tWnWrWfWv]$ 와 동일한 표현식이다. 맨 앞의 빈칸은 공백 문자(space)를 의미한다.

REPORT

`\S` - 화이트스페이스 문자가 아닌 것과 매치된다. `[\tWnWrWfWv]`와 동일한 표현식이다.

`\w` - 문자+숫자(alphanumeric)와 매치된다. `[a-zA-Z0-9_]`와 동일한 표현식이다.

`\W` - 문자+숫자(alphanumeric)가 아닌 문자와 매치된다. `[^a-zA-Z0-9_]`와 동일한 표현식이다.

■ re (regular expression) 모듈

- 파이썬 상에서 정규표현식을 사용하기 위해 모듈을 제공함.

Import 함수를 이용하여 아래와 같이 인용 할 수 있음.

```
Import re ## re 모듈 함수 이용을 위한 선언
```

```
P = re.compile('ab*') ##사용하여 정규 표현식(위 예시에서는 ab*)을  
컴파일한다
```

Method	목적
match()	문자열의 처음부터 정규식과 매치되는지 조사한다.
search()	문자열 전체를 검색하여 정규식과 매치되는지 조사한다.
findall()	정규식과 매치되는 모든 문자열(substring)을 리스트로 반환한다.
finditer()	정규식과 매치되는 모든 문자열(substring)을 반복 가능한 객체로 반환한다.

REPORT

◆ match의 경우 → 해당 예시에선 none을 반환함

```
pattern = r'apple'

text1 = "apple pie"

text2 = "sweet apple"

print(re.match(pattern, text1))  # <re.Match 객체 반환> (시작이 apple임)
print(re.match(pattern, text2))  # None (시작이 sweet임)
```

◆ match 컴파일 사용할 경우

```
import re

# 1. 패턴을 미리 컴파일하여 객체 생성

# 예: 'p'로 시작하고 'n'으로 끝나는 3글자 단어 패턴

p = re.compile('p.n')

# 2. 컴파일된 객체(p)의 메서드를 호출하여 조사 수행

### [match()] 문자열의 처음부터 조사

m1 = p.match("pan")    # 매치됨
m2 = p.match("japan")  # None (시작이 'j'이므로)

### [search()] 문자열 전체를 검색

s1 = p.search("pan")    # 매치됨
s2 = p.search("japan")  # 매치됨 (전체를 훑어서 'pan'을 찾아냄)
```

REPORT

[findall()] 매치되는 모든 문자열을 리스트로 반환

```
f = p.findall("pan pen pin")
```

결과: ['pan', 'pen', 'pin']

[finditer()] 매치되는 모든 문자열을 반복 가능한 객체로 반환

```
iter_obj = p.finditer("pan pen pin")
```

```
for item in iter_obj:
```

```
    print(item.group())
```

결과: pan, pen, pin 순차 출력

◆ search의 경우 → 패턴이 한 번이라도 나오면 그 위치를 반환함

```
pattern = r'apple'
```

```
text = "sweet apple pie"
```

match는 None을 주겠지만, search는 찾아냅니다.

```
print(re.search(pattern, text)) # <re.Match 객체 반환>
```

◆ findall의 경우 → 매치되는 모든 문자열을 리스트로 반환

```
pattern = r'\d+' # 숫자가 하나 이상 반복되는 패턴
```

```
text = "1st apple, 2nd banana, 3rd cherry"
```

```
result = re.findall(pattern, text)
```

```
print(result) # ['1', '2', '3']
```


REPORT

◆ finditer의 경우 → 매치되는 모든 결과를 반복 가능한 객체로 반환

```
pattern = r'\d+'
```

```
text = "1st apple, 2nd banana, 3rd cherry"
```

```
result_iter = re.finditer(pattern, text)
```

```
for match in result_iter:
```

```
    print(f"찾은 값: {match.group()}, 위치: {match.span()}")
```

```
# 출력:
```

```
# 찾은 값: 1, 위치: (0, 1)
```

```
# 찾은 값: 2, 위치: (11, 12)
```

```
# 찾은 값: 3, 위치: (23, 24)
```

---감사합니다.---